Artificial Intelligence 2025/26 - Main practical assignment

Reinforcement learning

Agustín Riscos Núñez, Jose María Moyano Murillo

1 Introduction to reinforcement learning

Reinforcement learning is a kind of machine learning in which an agent learns an optimal behaviour (i.e., a set of actions to take based on a given situation) by means of the interaction with the environment [1]. Such a learning is usually obtained through trial and error, while receiving rewards or penalties depending on the actions taken and the outcome achieved. Thus, it does not depend on a set of labeled examples (as most of the scenarios we have studied during the course). In this case, given a set of possible actions that may take place at specific states, the goal of the learning is to discover a policy that maximizes the final reward.

During the course, we have studied planning problems, assuming that the outcomes of applying an action is deterministic; but in many real-world problems, the actions might have a stochastic outcome or behavior, so that performing the same action several times would lead to completely different final scenarios. For example, in the blocks world problem, piling the block `x` on top of block `y` always leads to the `on(x,y)` state; while in a stochastic scenario, the arm could break down so that it does not release the `x` block, or the block falls and `x` ends up on the table and not on top of `y` block. Thus, while in the planning problems studied in Unit 4 during the course, the outcome was to obtain a plan, i.e., a sequence of actions to perform, in this kind of problems the goal is to obtain a **policy**, that indicates which action to apply depending on the current state.

The reinforcement learning problem can be stated as a finite Markov Decision Process (MDP) [1, 2]. MDPs are a formalization of sequential decision making, where the actions influence not only the immediate reward but also subsequent states, and therefore, future rewards.

Let us define some elements of the reinforcement learning problem that should be identified:

- **Policy:** Defines the agent's way of behaving, i.e., the action to take, at a given moment. Such a policy could be from a simple function or table, to a more extensive computational process.
- **Reward function** (or signal)**:** Defines the goal of a reinforcement learning problem. At each step carried out by the agent, the environment could send a reward value, which is to be maximized (in the long run) by the agent. Basically, such a function defines which are the *good* and *bad* events for the agent.
- **Value function**. While the reward indicates what is good in an immediate sense, the value function specifies what is good in the long run. Roughly speaking, the value of a given state is the expected amount of future reward obtained if starting from such state.

A common (and summarized) example of reinforcement learning problems is learning to play the *tic-tac-toe* game, which is further described in Section 1.5 in [1]. The goal of the game is to place `X`s and `O`s in a 3x3 grid, until one of them places three marks in a row.

- If we aim to learn a program to play *tic-tac-toe*, the possible actions to our problem would be to place a piece/mark in any of the 9 spaces of the board, while the applicable actions to a given state are to place the piece only in the uncovered spaces.
- In the simplest case, we could define the policy as a table, with as many entries as different states (i.e., different board combinations that could be achieved), and each of them having as next action the one that maximizes the state's value.
- The value of a given state could be defined as the estimated probability of winning the game if we go through that state. All the states could be initialized to a given value, such as 0.5 (or, maybe initializing the already winning boards, with 3-in-a-row with a value of 1; while the losing boards, with a 3-in-a-row for the opponent, with a value of 0).
- In order to let the program learn how to play, we let it play, for example, with a real player. At each time, it will look up the table given the current state and use the action that maximizes the highest value (or maybe introduce some randomness to increase exploration). At the end of the match, if the program won, the states it passed through should have their value increased, since they lead to a win, or if it lead to a lose (or a tie), the values of the corresponding states should be decreased.
- After a number of iterations, the value of each state would converge, so that the program could use the state values to create the **optimal policy**, i.e., the one that allows it to reach the maximum value.

2 Assignment description and objectives

The objective of this assignment is to design and implement a reinforcement learning algorithm able to learn how to play any of the two games described in the next section.

The algorithm must be completely designed and implemented by the student, without using external libraries that are for the specific purpose of reinforcement learning (but other useful external libraries could be used).

The **specific objectives** of the basic version of the assignment are:

1. Learn and research about reinforcement learning techniques and algorithms.
2. Design a reinforcement learning algorithm of your choice to solve the selected problem. The algorithm must be able to learn from scratch, by running a number of iterations, or any other kind of stopping criteria.
3. The program should implement the following functionalities:
    a) Let the user play versus the computer with the learned policy.
    b) Simulate a game between the model that is being trained and a "dummy player" driven by any kind of strategy designed and implemented by the student. More precisely, since it is not possible in practice to play thousands of games versus the computer during the learning process, the "dummy player" should be simulated, either with random movements or with a fixed game strategy designed and implemented by the student.
4. Run a series of experiments, at least, using different number of iterations for the learning process (or whatever other stopping criterion was considered), analyzing how the trained agent works.
5. Write a report containing the project documentation, explaining in detail all your design decisions concerning the reinforcement learning algorithm. The report should be written in the form of a scientific paper.
6. Make a short presentation about the implemented code, the decisions made during such process, and the results obtained during the experiments. The interviews for the defense of such work will be scheduled and announced after the submission deadline.

All these specific objectives should be accomplished, in order for the assignment to be accepted for evaluation. The code must be original and run with no errors, and it should include the possibility to replicate the experiments that have been carried out and analyzed in the report (either with a dedicated script or with detailed instructions). The report should follow the guidelines explained in Section 4.

The following additional requirements describe the **advanced version**, that is not mandatory, but will be evaluated as explained in Section 5.

- Implement different strategies (at least three) for the "dummy players" used during the training process, simulating different kinds of opponents.
- Repeat the experiments using different values for the parameters (e.g. different reward functions), and different settings for the game.
- Display detailed statistics about the policy analyzing, for example, how the agent behaves depending on the current state.

3 Description of the games

This is a short description of the two possible games that you may choose to work on. **Important:** The selection of the game will be managed by means of the corresponding assignment on Moodle. Register your choice before starting to work on the assignment in order to make sure that it is still available.

3.1 Dice game - Pig

Pig is a simple dice game which, in its basic form, can be played using only one dice, and from 2 to any number of players; in our case, we will consider the two-players version. The goal is to reach an established number of points, being usually 100.

The rules are simple. First, the starting player (and the order, if there are more than two) is decided randomly. In turns, each player rolls the dice and scores as many points as shown on the dice; providing that he/she does not roll a 1. After each roll, the player may continue rolling the dice and accumulating points, or end his/her turn, adding the accumulated points to the score. However, in the case the player rolls a 1, he/she loses all the accumulated points in this turn, and passes the dice to the next player. The game ends when one player accumulates at least 100 points.

There are different variations of the simple pig game. In the *six-trigger* pig version, when a player rolls a 6, adds it to the accumulated score as in the base one, but is forced to continue rolling at least once more in the turn. On the other hand, in the *two-dice* pig version, two dices are rolled each time; obtaining a single 1 in any of the dices ends the turn as in the base version; but if both dices roll a 1, the complete accumulated score of the player during the whole game is lost.

If this option is chosen, the reinforcement algorithm must learn how to play the pig dice game, in any of the two aforementioned variations (not the base one).

3.2 Race game - The floor is slime!

"The floor is slime!" is a simple race game which includes a random component concerning the floor tiles, we will consider the two-players version. The goal is to be the first to reach the final tile after an established number of successful jumps, being initially set to 10.

The tiles are separated and arranged in a row, in such a way that the players start the game located on the first tile, and the last tile is the goal. At each moment, the tiles can be either dry or full of slime, but after each round of the game (that is, after all players have made their move), each empty tile has a 50% chance to flip to the opposite state. A player wins the game if he/she jumps through the tiles and is the first to safely reach the last tile. At each step, the players can choose one of two possible moves: jump or stay. If the player decides to jump, there is a probability that he/she will safely land on the next tile, but there is also a possibility of falling out of the tile, in which case the player needs to go back to the first tile. Obviously, the chances of safely landing on a dry tile are higher than if the tile is slippery because it is full of slime.

Figure 1: Illustrative image for the game, generated using Gemini

There are different variations of the simple race game. In the *small-tiles* version, it is forbidden that two players share the same tile, so when a player lands on a tile that is already occupied, then one of them (or both) will fall out of the tile. On the other hand, in the *long-jumps* version, we add a third possible move: players can try to advance two tiles in one jump, but the chances of safe landing should be lower than those of the single-tile jump, both for the dry and the slimy case.

If this option is chosen, the reinforcement algorithm must learn how to play "the floor is slime!" race game, in any of the two aforementioned variations (not the base one). You are free to choose the values for the probabilities of falling/safe landing on each of the cases.

4 Submission

The report must follow the IEEE conference template[1], either in *.doc* or LaTeX *.tex* format. Anyway, the submitted document must be in PDF format.

The paper should be at least 6 pages long, and the general structure of the paper should be as follows: first, an introduction explaining the main objective, including a brief background on the subject of the work and the methods used (including bibliographic references); then describe the structure of the proposal, the design decisions that were made during its development (also mentioning elements that were initially considered but subsequently discarded, if any), as well as the methodology followed to implement it (never include code in the report, but pseudocode if considered); then detail the experiments carried out, analyzing the results obtained. Finally, the document must include a conclusions section, and a bibliography that includes not only the references cited in the introduction section, but also any documents consulted during the realization of the work (including web references to pages or repositories). It is important that the document support and justifies both the design decisions made throughout the process, as well as the conclusions drawn from experimental results.

It is necessary to meet the requirements described in Section 2 (each and every of them, at least in its basic version) for the work to be evaluated. A single compressed *.zip* file must be submitted to the corresponding task in the course webpage, including:

- **The document in PDF format**, that must follow the aforementioned requirements.
- **A folder with the source code.** Within this folder, there must be a README.txt file that summarizes the structure of the source code and indicates how to run the algorithm, including usage examples and how to reproduce the different experiments. It is important the coherence of this file with the defense.

The day of the defense will be announced to each student with enough time, once the assignment deadline has passed. On that day, a small 10-minutes presentation (using slides in PDF, PowerPoint, or similar resources) should be made. This presentation will roughly follow the same structure of the paper, but with a special emphasis on the decisions made through the design and implementation process and the results of the experimental study as well as their critical analysis. Following, for about another 10 minutes, the professor will ask questions about the work, which may include both the document and the source code.

[1] `https://www.ieee.org/conferences/publishing/templates.html`

5 Evaluation

The evaluation of the assignment will take into account the following criteria.

- **Source code (0.5 points):** the clarity and good programming style[2], correctness and efficiency in implementation, quality of comments, and the correspondence between the algorithm explanation in the report and the code will be valued. In any case, the code must be original, and the work will not be evaluated if any copied or downloaded code from the internet is detected.
- **Scientific report (0.5 points):**
  - Appropriate use of language and the overall report style.
  - The preliminary research work done will be evaluated as long as it is clear that the research is understood.
  - Clarity of explanations, reasoning of decisions, definition of the different parts of the algorithm (such as reward and value functions, policy, memory, etc.), and the analysis and presentation of results in the Experiments and Conclusions sections.
- **Ease of use and experimentation (0.5 points):** the number and variety of experiments performed will be evaluated. It will be also considered the study on the performance for different values of the arguments (such as the number of rounds in the training process). Usability is also evaluated when reproducing the same or new experiments.
- **Advanced version (0.75 points):** the number, variety and complexity of "dummy players" implemented will be evaluated. It will be also considered the study of different versions of the selected game (different reward functions, original extensions or modifications of the rules, etc). Usability is also evaluated when reproducing the same or new experiments.
- **Presentation and defense (0.75 points):** the clarity of the presentation, the good explanation of the contents of the work, and the answers to the questions asked by the professor will be considered.
  Important: If the student is unable to defend his/her work correctly during the defense, raising reasonable concerns about potential learning deficiencies and/or about the authorship of the submitted material, then the entire assignment will be graded with a 0.
- **Improvements:** any improvement or expansion in any aspect that has not been considered in this document might be evaluated with up to 0.5 extra points, without exceeding the maximum total of 3 points in any case.

[2] `https://docs.python-guide.org/writing/style/`

6 Autorship and use of generative AI

Upon submission of the assignment, the student affirms that he/she is the original author of the work.

The use of generative AI and similar tools are not prohibited, but the students must always be aware of and understand the basis for the implementations they have made. We are aware that AI tools could be useful to obtain or contrasting ideas, or even assisting with implementation, but as the main objective of the work is to learn, it is therefore mandatory to fully comprehend all the decisions made in the algorithm as well as its implementation. If such tools are used, it should be specified which ones and for what purpose in a specific section of the submitted paper. Besides, it is not recommended to rely solely on AI to learn about reinforcement learning or to obtain ideas for algorithm proposals.

Any plagiarism, code sharing, use of material that is not original and does not properly cite the source, or misuse of AI tools **will automatically result in a zero grade for the course for all students involved**. Therefore, these students will not receive any grades for the current call. This is without prejudice to any disciplinary action that may be taken.

References

[1] Richard S Sutton and Andrew G Barto. *Reinforcement learning: An introduction*. MIT press Cambridge, second edition, 2018.

[2] Chelsea C White III and Douglas J White. Markov decision processes. *European Journal of Operational Research*, 39(1):1–16, 1989.